UTS COMP6991031 · JAWABAN SOAL 1 (LO 1)

BookVerse

Analisis projek dan pemilihan model layanan cloud untuk BookVerse - aplikasi rekomendasi buku berbasis TF-IDF yang berjalan sebagai aplikasi multi-tier terkontainerisasi.

Nama: _____ · NIM: _____ · 10 November 2026

KATEGORI AI · TF-IDF + COSINE SIMILARITY · DOCKER

LAPORAN

Mata Kuliah	COMP6991031 - Cloud Computing (Ujian Tengah Semester)
Projek	BookVerse - AI Book & Novel Recommender
Kategori	AI (aplikasi web dengan mesin rekomendasi)
Nama / NIM	_____ / _____
Tanggal	10 November 2026

Jawaban Soal 1 (LO 1)

Analisis Projek & Pemilihan Model Layanan Cloud - 30 poin (a: 8 poin, b: 10 poin, c: 12 poin)

Soal 1(a) - Profil dan Alasan Pemilihan Projek (8 poin)

1) Nama, kategori, masalah, dan target pengguna

BookVerse - aplikasi web kategori **AI** (contoh soal: rekomendasi buku). Mesinnya *content-based filtering*: setiap buku dijadikan vektor TF-IDF (genre + penulis + sinopsis) lalu dicocokkan dengan *cosine similarity*, murni CPU.

Masalah: *choice overload* - pembaca bingung memilih bacaan berikutnya karena pencarian katalog hanya berbasis kata kunci dan tidak memahami selera. **Target pengguna:** pembaca kasual dan mahasiswa (internasional + Indonesia) yang ingin rekomendasi cepat tanpa membuat akun.

Rp 0	**4**	**55**	**6**
target biaya operasional	container Docker	buku dalam katalog	rekomendasi per prediksi

2) Satu fitur inti (MVP)

Rekomendasi cerdas dari judul atau deskripsi bebas (**POST /api/predict**). Mode judul: buku yang disukai dibandingkan ke seluruh katalog; mode teks: deskripsi bebas seperti “magic school adventure with friendship” di-vektorisasi dengan kosakata yang sama. Hasilnya enam buku teratas plus skor kemiripan. Fitur lain (katalog 55 buku, riwayat, cache Redis) hanya pendukung.

3) Motivasi pribadi dan kaitan karier

Saya pembaca aktif yang sering lama memilih bacaan berikutnya, dan ingin membuktikan TF-IDF klasik sudah cukup untuk masalah ini. Saya menargetkan karier **Backend Engineer**: projek ini melatih inti pekerjaan backend - REST API (FastAPI), SQL/ORM (PostgreSQL), caching (Redis), Docker - sedangkan mesin AI-nya sengaja ringan agar fokusnya pada mengemas model menjadi layanan yang andal dan murah.

4) Requirement: teknis, biaya, kepatuhan, karakteristik NIST

Aspek	Requirement singkat
Teknis	Empat container Docker Compose (nginx, FastAPI + scikit-learn, PostgreSQL 16, Redis 7); CPU-only tanpa GPU; healthcheck + named volume; REST/JSON.
Biaya	Target **Rp 0**: semua komponen open source, tanpa API key; sampul buku dari CDN publik; di cloud memakai free tier.
Kepatuhan	Hanya katalog bibliografis publik + riwayat rekomendasi **anonim** (tanpa akun/login) sehingga tidak ada data pribadi dalam arti UU No. 27/2022 - desain tanpa akun adalah minimisasi data. Bila nanti ada fitur akun: perlu persetujuan eksplisit (Pasal 20), hak akses/penghapusan (Pasal 5-13), enkripsi (Pasal 35), dan lapor kebocoran 3 × 24 jam (Pasal 46).
Karakteristik NIST	***Measured service***: konsumsi CPU/memori/jaringan terukur dan bisa diberi ambang peringatan agar target Rp 0 tidak terlampaui.

Tabel 1. Ringkasan requirement projek BookVerse.

Soal 1(b) - Perbandingan Lima Model Layanan (10 poin)

Tingkat abstraksi = berapa lapis tumpukan teknologi yang disembunyikan penyedia; semakin tinggi abstraksi, semakin besar porsi penyedia dalam tanggung jawab bersama (*shared responsibility*).

Model	Tingkat abstraksi	Dikelola penyedia vs saya	Contoh	Cocok untuk BookVerse?
IaaS	Terendah - infrastruktur virtual (VM, storage, jaringan).	Penyedia: fisik, virtualisasi, jaringan. Saya: OS, patch, runtime, aplikasi, data.	AWS EC2	Bisa (Docker dipasang sendiri), tetapi terlalu berat operasional untuk satu pengembang - tidak dipilih.
PaaS	Menengah - platform runtime siap pakai.	Penyedia: OS, patching, scaling platform. Saya: kode, konfigurasi, data.	Render / Railway	**Paling cocok:** cukup push image Docker, DB dan cache terkelola, saya fokus ke backend.
SaaS	Tertinggi - aplikasi jadi siap pakai.	Penyedia: seluruh aplikasi. Saya: hanya data dan konfigurasi pemakaian.	Google Workspace	Tidak cocok - BookVerse justru aplikasi yang saya bangun; SaaS hanya alat pendukung.
BaaS	Tinggi di lapisan backend - layanan ber-SDK (auth, DB, storage).	Penyedia: backend generik + SDK. Saya: logika frontend dan aturan bisnis.	Firebase / Supabase	Cocok sebagian - PostgreSQL terkelolanya bisa dipakai, tetapi mesin TF-IDF + Redis tetap butuh compute sendiri.
FaaS	Sangat tinggi - fungsi event-driven tanpa server.	Penyedia: server dan scaling per pemanggilan. Saya: kode fungsi + dependensi.	AWS Lambda	Kurang cocok sebagai host utama - matriks TF-IDF (state) dibangun ulang tiap cold start; hanya pas untuk job kecil.

Tabel 2. Perbandingan IaaS, PaaS, SaaS, BaaS, dan FaaS untuk BookVerse. Catatan: NIST SP 800-145 hanya mendefinisikan IaaS/PaaS/SaaS; BaaS dan FaaS adalah turunan pasar di atas PaaS.

Soal 1(c) - Model Deployment dan Rekomendasi (12 poin)

1) Perbedaan public, private, hybrid, dan multi-cloud

Model	Definisi singkat	Implikasi utama
Public	Infrastruktur milik penyedia, dipakai bersama (multi-tenant) lewat internet.	Mulai cepat tanpa capex; skalabilitas besar; kontrol lewat konsol/konfigurasi.
Private	Cloud khusus satu organisasi (on-premise maupun hosted).	Isolasi dan kepatuhan maksimal; butuh capex dan tim operasional sendiri.
Hybrid	Public + private terintegrasi dengan manajemen terpadu.	Beban sensitif di private, lonjakan elastis di public; kompleksitas menengah.
Multi-cloud	Dua penyedia public atau lebih dipakai paralel.	Menghindari lock-in dan risiko satu penyedia; operasional paling berat.

Tabel 3. Empat model deployment cloud.

2) Rekomendasi service model + deployment model

> **Rekomendasi**
>
> PaaS berorientasi container di **public cloud**: jalankan image Docker BookVerse di Render / Fly.io / Google Cloud Run / AWS App Runner, plus PostgreSQL dan Redis terkelola.

Teknis: seluruh tumpukan sudah terkontainerisasi - image tinggal didorong ke registry lalu dijalankan; healthcheck Compose dipetakan ke health probe platform; TLS, patching OS, dan autoscaling jadi urusan penyedia; scikit-learn di CPU cukup instance terkecil.

Model harga	Cara kerja	Posisi dalam rencana BookVerse
Free tier	Kuota gratis bulanan (komputasi, memori, permintaan).	Tahap awal - target Rp 0 tercapai; mis. Cloud Run gratis 2 juta permintaan/bulan.
Pay-as-you-go	Tagihan per pemakaian aktual, tanpa komitmen.	Jalur utama saat pengguna nyata datang; biaya mengikuti trafik.
Reserved	Komitmen 1-3 tahun untuk kapasitas tetap, diskon besar.	Belum relevan; hanya untuk beban stabil 24/7 (mis. DB produksi) di masa depan.
Spot instance	Kapasitas sisa berdiskon besar, dapat dihentikan sewaktu-waktu.	Tidak untuk API interaktif; hanya pekerjaan batch (mis. rebuild indeks TF-IDF).

Tabel 4. Pemetaan model harga terhadap tahapan projek.

Kepatuhan: data yang naik ke cloud tetap bukan data pribadi (katalog publik + riwayat anonim) sehingga risiko UU No. 27/2022 minimum; region dipilih terdekat pengguna (Singapura/Jakarta) demi latensi. Bila kelak ada fitur akun: pilih penyedia dengan perjanjian pemrosesan data dan enkripsi *at-rest*, atau pindahkan modul data ke private cloud kampus sehingga arsitektur menjadi **hybrid**.

3) Dua trade-off utama

> **Trade-off 1 - Vendor lock-in**
>
> Semakin lama terikat layanan managed satu penyedia (DB, cache, fitur platform), semakin mahal pindah. Mitigasi: protokol terbuka (wire protocol PostgreSQL/Redis) dan image Docker + docker-compose.yml sebagai sumber kebenaran topologi.

> **Trade-off 2 - Cold start dan batas free tier**
>
> Instance gratis “tidur” saat idle sehingga permintaan pertama lambat - diperparah TF-IDF dibangun ulang saat startup - dan kuota bisa terlampaui diam-diam. Mitigasi: peringatan anggaran, pantau pemakaian (*measured service*), bayar satu instance minimal saat trafik nyata.

4) Bagaimana Docker memudahkan perpindahan

Image = unit portabilitas: build once, run anywhere - dependensi (Python, virtualenv, nginx) terkunci di dalam image sehingga hilang masalah “di laptop saya jalan”. **docker-compose.yml = cetak biru topologi** (service, network, volume, healthcheck) yang bisa diterjemahkan ke platform mana pun. **Registry** (Docker Hub) membuat penyedia lain menarik image dalam hitungan menit, jadi pindah penyedia atau multi-cloud cukup mengganti tujuan pull tanpa rebuild. Image multi-stage yang ramping juga memperpendek cold start.

Referensi

Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing* (NIST SP 800-145). NIST. https://doi.org/10.6028/NIST.SP.800-145

Republik Indonesia. (2022). *UU Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi*. Lembaran Negara RI Nomor 157.

Docker, Inc. (2026). *Docker overview*. https://docs.docker.com (diakses 10 November 2026).

Google Cloud. (2026). *Cloud Run pricing*. https://cloud.google.com/run/pricing (diakses 10 November 2026).